DESARROLLO DEL SOFTWARE



Steve

Desarrollador Software

Ciclo Vida Desarrollo Software (SDLC)







Requerimientos Cliente

1. PLANIFICACIÓN
2. ANÁLISIS REQUISITOS
3. DISEÑO
4. IMPLEMENTACIÓN
5. TESTEO
6. DESPLIEGUE & MANTENIMIENTO

El ***ciclo de vida del desarrollo de software (SDLC)*** se puede concebir como una hoja de ruta que va guiando todas las fases del proceso tecnológico. Sin embargo, abarca diferentes modelos, por lo que si queremos aplicar uno, no solo hay que conocer sus características, sino cómo se adaptan a cada proyecto según sean las necesidades.

Metodologías Tradicionales (Secuenciales)

Se enfocan en una planeación estricta y detallada desde el inicio del proyecto. Las fases avanzan en línea recta y no permiten retrocesos fáciles una vez que una etapa ha terminado.

- **Modelo en Cascada (Waterfall):** Organiza las tareas de arriba hacia abajo en etapas sucesivas (análisis, diseño, programación, pruebas). Se avanza a la siguiente fase solo al concluir la anterior de forma rigurosa.
- **Modelo en Espiral:** Combina fases de diseño en cascada con iteraciones frecuentes enfocadas primordialmente en el **análisis de riesgos** y control de presupuesto.
- **Modelo de Prototipos:** Crea un borrador o modelo rápido del software para que los usuarios interactúen con él y den opiniones tempranas que mejoren los requerimientos.

Metodologías Ágiles (Iterativas)

Se adaptan de manera constante a los cambios y priorizan la colaboración frente a la documentación rígida. El trabajo se fracciona en partes pequeñas para entregar valor de forma continua.

- **Scrum:** Divide el proyecto en ciclos cortos llamados *sprints* (de una a cuatro semanas). Cuenta con roles definidos como el *Product Owner* y el *Scrum Master* para asegurar entregas parciales funcionales.
- **Kanban:** Utiliza un tablero visual para representar el flujo de las tareas en tiempo real. Ayuda a limitar el trabajo en curso y optimizar la flexibilidad del equipo.
- **Extreme Programming (XP):** Busca elevar la calidad del código y la capacidad de respuesta ante requerimientos cambiantes mediante prácticas estrictas de ingeniería y pruebas continuas.

Enfoques Modernos e Híbridos

- **DevOps:** Une el desarrollo de software con las operaciones de infraestructura. Promueve la mejora continua, la automatización y la retroalimentación sin interrupciones durante todo el ciclo de vida del sistema

¿Qué es el ciclo de vida del software (SDLC)?

El ciclo de vida del software (SDLC) es un proceso que se sigue para desarrollar y mantener el software de manera efectiva. Este ciclo se compone de varias fases que se llevan a cabo secuencialmente, con el fin de asegurarse de que el software cumpla con los requisitos y objetivos establecidos al principio del proyecto. Las fases del ciclo de vida del software incluyen la planificación, análisis, diseño, implementación, pruebas, despliegue y mantenimiento.

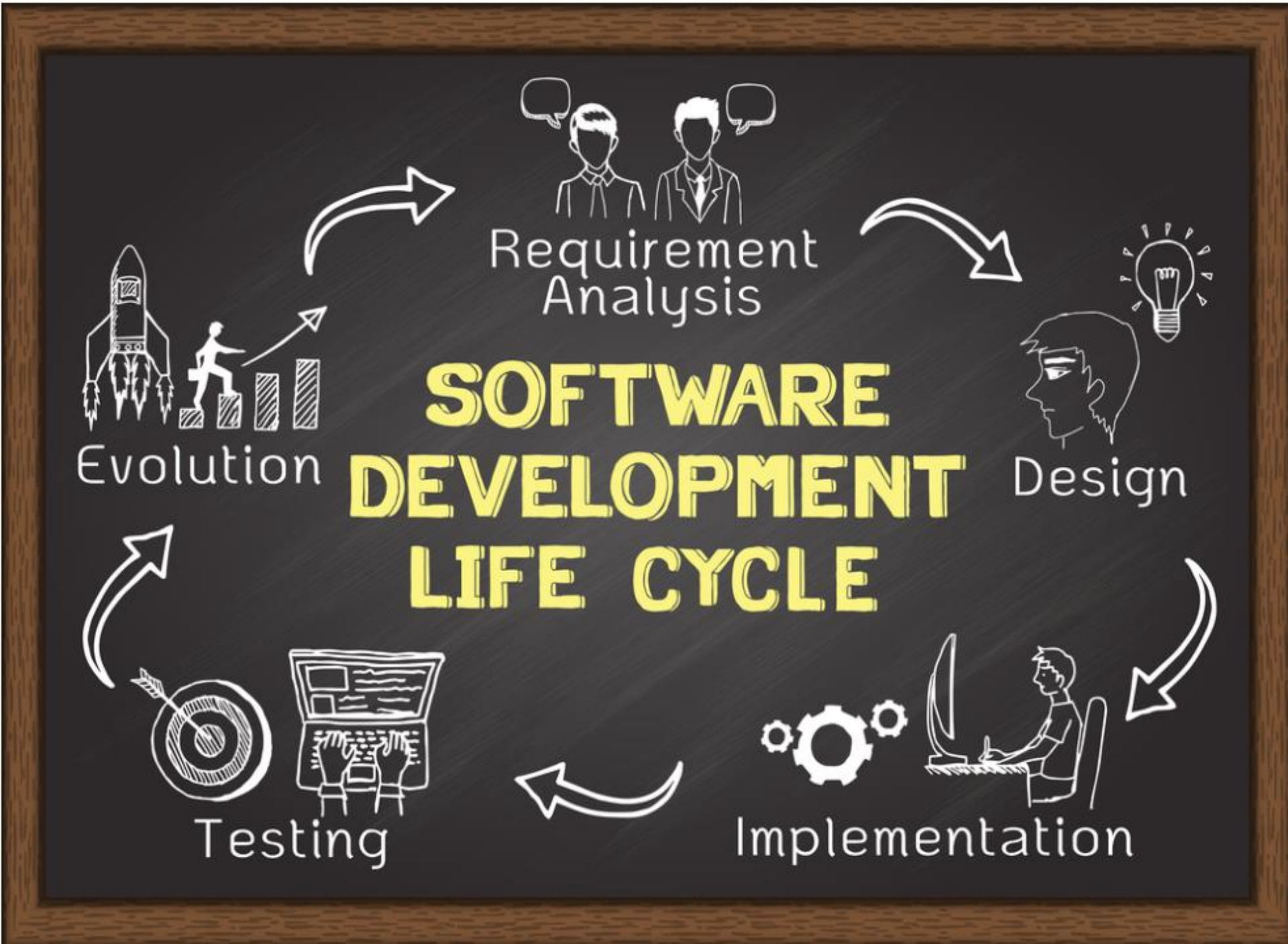
Objetivos principales del SDLC

El ciclo de vida del desarrollo de software se puede considerar como una referencia para facilitar el proceso de creación del producto. El objetivo principal es asegurarse de que este tendrá la calidad esperada y se ajustará a las necesidades del usuario final. Sin embargo, también se utiliza para optimizar los recursos que hay disponibles, así como para reducir los riesgos que puedan surgir durante el desarrollo.

Beneficios para equipos de desarrollo y empresas

Las ventajas de utilizar el SDLC son múltiples, aunque cambien en función del punto de vista que se tome.

- Para el equipo de desarrollo, es una guía que mejora la coordinación, la comunicación y el control del proyecto.
- Aumenta la eficiencia, se pueden identificar fallos en las primeras fases y se facilita la adaptación a los cambios.
- Para las empresas, con el SDLC, es más fácil ajustarse al presupuesto inicial y a los plazos.
- También se reducen los fallos graves y aumenta la satisfacción del cliente.


Requirement Analysis
Design
Implementation
Testing
Evolution
SOFTWARE DEVELOPMENT LIFE CYCLE

Etapas SDLC

- Análisis
- Diseño
- Desarrollo
- Pruebas
- Despliegue
- Mantenimiento

Etapa de Análisis

Es el proceso de **investigar un problema** que se quiere resolver.

- Identificar un problema
- Definir el problema
- Identificar los componentes principales
- Requerimientos

La identificación de los componentes principales en el análisis de sistemas de software es el proceso de dividir un sistema grande en piezas clave para entender su estructura, sus funciones y cómo se relacionan entre sí.

¿Por qué es importante?

- Facilita la comprensión: Permite ver un sistema complejo como un conjunto de partes más pequeñas y fáciles de entender.
- Organiza el trabajo: Ayuda a los desarrolladores a saber qué se debe programar o ajustar.
- Mejora el mantenimiento: Permite encontrar y reparar fallas en una parte específica sin afectar a todo el programa

Ejemplos de componentes principales

- Interfaces de usuario: Las pantallas, botones y menús con los que interactúa la persona.
- Módulos de lógica: Los bloques de código que procesan los datos y toman decisiones según las reglas del negocio.
- Bases de datos: El espacio seguro donde se guarda y organiza toda la información del sistema.
- Servicios externos o APIs: Las conexiones que permiten que el software hable con otros programas o servidores.

Etapa de Diseño

Diseñar el software con la ayuda de toda la **información recolectada** en la etapa de análisis.



Etapa de Desarrollo

Con modelos creados en la etapa de diseño comenzamos la **codificación**.



Etapa de Pruebas

- **Componentes** que se integran al sistema **cumplen** con los requerimientos del **etapa de diseño**.
- Sin errores en la integración.



Etapa de Implementación

En esta etapa de pone **a disposición del cliente** el software desarrollado.



Etapa de mantenimiento

Consisten en **corregir problemas** del software y generar una **nueva versión**

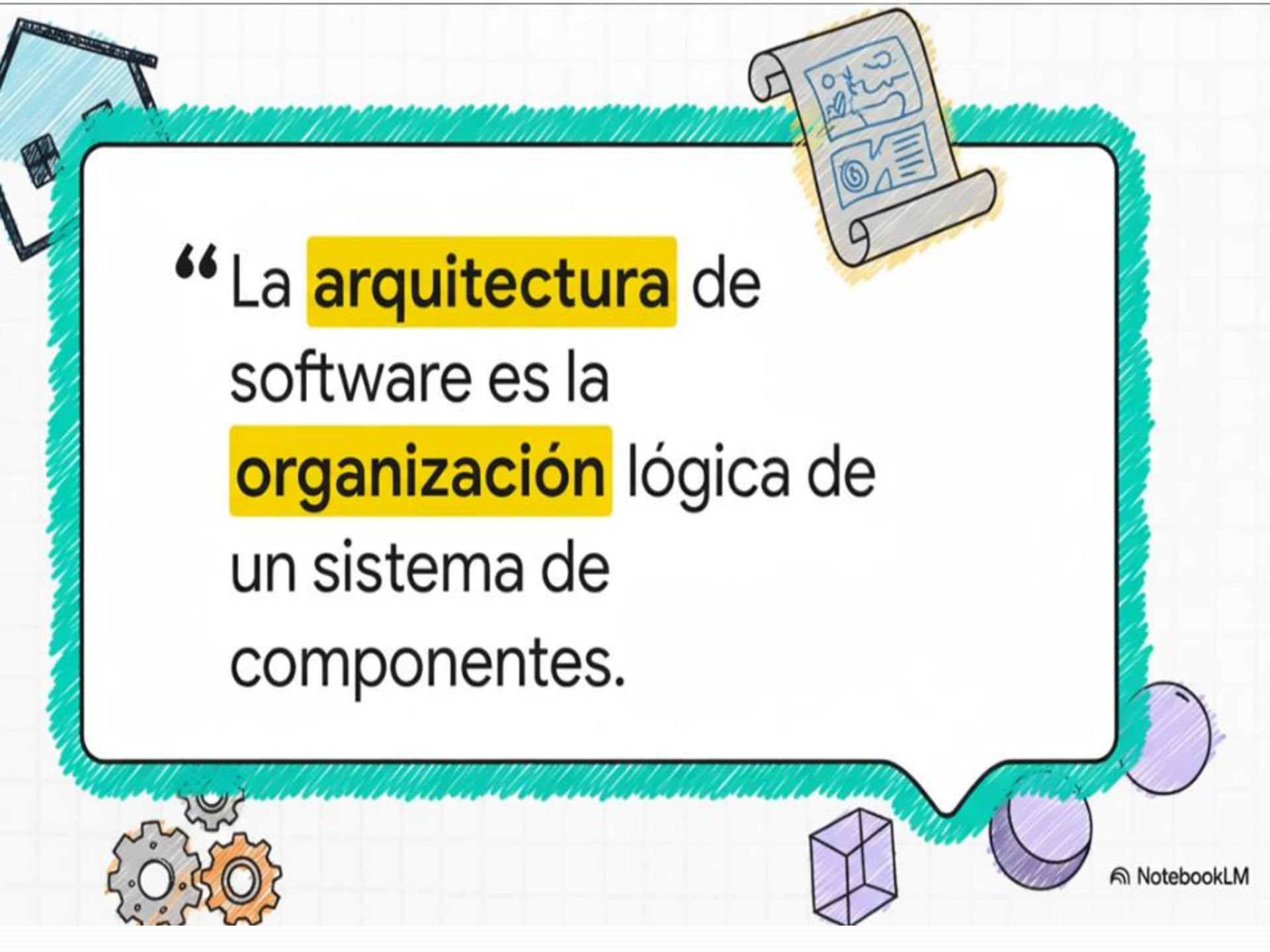
.



Análisis y Desarrollo de Software

- Análisis	Identificación del problema o necesidad	Software	Requerimientos
- Diseño	Arquitectura de Software	Diagramas de flujo, lenguaje de modelado (uml)	Tecnologías y Lenguajes
- Desarrollo	Codificación		Front end (Html + Css + Javascript + React)
			Back end (python)
			BD (MysQl, Firebase)
-Prueba	Unit test - Prueba unitaria		Figma (UX | UI)
- Mantenimiento	Manual del sistema, Documentación, Legal		


3
Arquitectura de software
Cimientos y armazón
NotebookLM


“La arquitectura de software es la organización lógica de un sistema de componentes.
NotebookLM

Arquitectura en capas

